

Hello and thank you for using fusion. It is my privilege to introduce you to my programming language.

Fusion acts as an interoperability layer between software ecosystems. Fusion is a programming language designed around ecosystem interoperability.

You can consider fusion a **unified software development platform with a language attached.**

When you install fusion (vanilla or basic), you get something like this:

Fusion installation

- fusion compiler/interpreter
- toolchain orchestrator
- »**foundry**« the package manager
- static analyser(it suggests, you should move functions to another prog language)
- interoperability model(offering 5 layers of interoperability)

There are 2 types of plugins(THAT STILL NEED TO BE BUILT), the first type are **tooling/ergonomic in nature**, they make development/programming more efficient, pleasant and easier to maintain. These plugins include SQLite, UI framework, cloud builds, REPL+jupyter, JIT+AOT, actors system(from erlang/elixir) and more.

The second type **increase the access and expands the reach of fusion**. It allows fusion to treat foreign programming languages as resources to be exploited and expanded. By integrating or embedding foreign source code files, fusion can compile/build files written in another programming language with minimal overhead, no rewrite needed, let alone installing and maintaining a foreign programming language.

Ask yourself, if you can build it in Fusion, if the answer is no, which ecosystem is the best implementation for this specific capability?

Your knowledge should be transferable to fusion, regardless which programming language you use.

I am pushing Fusion toward being an **orchestration/interoperability language**, rather than merely another general-purpose language.

The language is the interface.

The compiler is the execution mechanism.

Foundry is the ecosystem coordinator(package manager).

Interoperability is the reason the whole thing exists.

Fusion may suggest to you that a function written in fusion would be faster or take less memory in another programming language like C and suggest to move it to a C file.

The Fusion Programming Language

Language Guide and Reference

-----qualities and characteristics-----

Fusion has:

- static typing with type inference,
- variables are mutable by default,
- with strong typing.

Fusion is meant to be **extremely modern** and beginner friendly.

You can make comments with `<///>` or `<#>`, for multi-line comments use `</**/>`.

You can use indentation or braces, whatever you like. **The main function is required.**

To use indentation write main:

To use braces write main{

Remember there is no mixing within the same file! Indentation is the official style.

main:

```
value = input()      // reading input
```

```
x = 42 as float      // converting to float
```

```
y = num::from(32)    // the value of y will be 42
```

```
fn from_num(value) -> num:      // user defined converter
    return value + 10
```

```
//      try conversion returns an option
```

```
z = num::try("42")
```

```
print(z)             // Some(42)
```

```
s = num::try("not-a-number")
```

```
print(s)             // None
```

-----Rust and zig-----

The GC is responsible for ordinary memory, while »defer« is responsible for deterministic cleanup of resources that need it. I'm deliberately separating **memory lifetime** from **resource lifetime**.

That gives Fusion something closer to:

GC for memory + RAII-like deterministic cleanup for resources

Fusion has many features borrowed from rust such as **rust-style: match, enums, structs, impl, traits, iterators.**

There is no need for dictionaries, all you need is **a growable array and a hash map** for everything.

Fusion has many operations for growable arrays, hash maps, strings and generic rust-style iterators.

- growable arrays(+operations): create, add, remove_last, access, modify, get_length, iterate, clear, insert_at_index, remove_at_index.

```
numbers = [1, 2, 3]
numbers.add(4)
numbers.insert_at_index(1, 9)
numbers.modify(0, 7)
numbers.access(0)
numbers.get_length()
numbers.remove_at_index(1)
numbers.remove_last()
numbers.clear()
numbers.get_length()
```

- hash map(+operations): create, insert, get, remove, contains_key, iterate, clear, iterate, keys, values.

```
m = hashmap()
m.insert("one", 1)
m.insert("two", 2)
m.contains_key("one")
m.get("two")
m.keys()
m.values()
m.remove("one")
m.clear()
m.get_length()
```

- iterators(+operations): map, filter, enumerate, collect, find, any, all, fold.

```
numbers = [1, 2, 3]
doubled = numbers.iterate().map(double).collect()
filtered = numbers.iterate().filter(even).collect()
found = numbers.iterate().find(even)
numbers.iterate().any(even)
numbers.iterate().all(even)
total = numbers.iterate().fold(0, add)
```

- string operations(+operations): create, concatenate, substring, find, replace, split, trim, to_uppercase, to_lowercase, starts_with, ends_with, contains, length, iterate.

```
text = " Hello Fusion "  
text.trim()  
text.to_uppercase()  
text.to_lowercase()  
text.substring(2, 7)  
text.find("Fusion")  
text.replace("Fusion", "World")  
text.split(" ")  
text.starts_with(" He")  
text.ends_with(" ")  
text.contains("Fusion")  
text.length()
```

It also has: async/await, Filesystem, Command-line arguments, Process/system functionality, Time and dates, Randomness, networking, serialisation, formating, encoding, logging.

```
date_utc(0)  
hex_encode("Fusion")  
hex_decode("46757369666e")  
base64_encode("Fusion")  
base64_decode("RnVzaW9u")  
format("value={0}", [42])  
serialize([1, 2, 3])  
deserialize("[1,2,3]")  
random_int(7, 7)  
log_info("ready")
```

examples

```
main:
  foo()
```

```
fn foo():
  return
```

```
main:
  value = input()
  print(value)
// reading input and printing it
```

```
main:
  x = 42 as float
  print(x)
```

```
main:
  print(foo())

fn foo():
  return 42
//expected output 42
```

```
fn from_num(value) -> num:
  return value + 10
```

```
main:
  x = num::from(32)
  print(x)
```

```
fn identity<T>(value: T) -> T:
  return value
```

```
main:
  result = identity<num>(42)
  print(result)
```

```
main:
  x = num::try("42")
  print(x)
  y = num::try("not-a-number")
  print(y)

//expected output: "Some(42)", "None"
```

```
fn identity<T>(value: T) -> T:
  return value

main:
  result = identity<string>("hello")
  print(result)
```

```
fn identity<T>(value: T) -> T {
  return value
}

main {
  result = identity<num>(42)
  print(result)
}
```

```
struct Person:
  name: string
  age: num

main:
  person = Person("Alice", 30)
  print(person.name)
  print(person.age)
```

```
struct Point {
  x: num
  y: num
}

main {
  point = Point(10, 32)
  print(point.x + point.y)
}
```

```
struct Person:
  name: string

trait Greeter:
  fn greet(person: Person) -> string

impl Greeter for Person:
  fn greet(person: Person) -> string:
    return "hello " + person.name
main:
  p = Person(name: "Fusion")
  print(p.greet())
```

```
main:
  file = open("{}")
  defer file.close()
  file.write("hello fusion")
```

```
enum State:
  Ready
  Failed(string)

main:
  state = State::Failed("broken")
  match state:
    State::Ready => print("ready")
    State::Failed(message) => print(message)

//expected output: broken
```

```
main:
  numbers = [1, 2, 3]
  numbers.add(4)
  numbers.insert_at_index(1, 9)
  numbers.modify(0, 7)
  print(numbers.access(0))
  print(numbers.get_length())
  print(numbers.remove_at_index(1))
  print(numbers.remove_last())
  numbers.clear()
  print(numbers.get_length())
```

```
// expected output: 75940
```

```
fn double(x: num) -> num:
  return x * 2
```

```
fn even(x: num) -> bool:
  return x == 2
```

```
fn add(acc: num, x: num) -> num:
  return acc + x
```

```
main:
  numbers = [1, 2, 3]
  doubled = numbers.iterate().map(double).collect()
  print(doubled)
  filtered = numbers.iterate().filter(even).collect()
  print(filtered)
  found = numbers.iterate().find(even)
  print(found)
  print(numbers.iterate().any(even))
  print(numbers.iterate().all(even))
  total = numbers.iterate().fold(0, add)
  print(total)
```

```
//expected output: [2, 4, 6], [2], Some(2), true, false, 6
```

main:

```
m = hashmap()
m.insert("one", 1)
m.insert("two", 2)
print(m.contains_key("one"))
print(m.get("two"))
print(m.keys())
print(m.values())
print(m.remove("one"))
m.clear()
print(m.get_length())
```

//expected output: true, Some(2), [one, two], [1, 2], Some(1), 0

main:

```
text = " Hello Fusion "
print(text.trim())
print(text.to_uppercase())
print(text.to_lowercase())
print(text.substring(2, 7))
print(text.find("Fusion"))
print(text.replace("Fusion", "World"))
print(text.split(" "))
print(text.starts_with(" He"))
print(text.ends_with(" "))
print(text.contains("Fusion"))
print(text.length())
```

//expected output: "Hello Fusion",
" HELLO FUSION ",
" hello fusion ",
"Hello",
"Some(8)",
" Hello World ",
"[, , Hello, Fusion, ,]",
"true",
"true",
"true",
"16"

```
async fn answer() -> num:  
    return 42
```

```
main:  
    value = await answer()  
    print(value)
```

```
// expected output: 42
```

```
main:  
    write_file("{} ", "hello fusion")  
    print(file_exists("{}"))  
    print(read_file("{}"))  
    remove_file("{}")
```

```
// expected output: "true", "hello fusion"
```

```
main:  
    print(date_utc(0))  
    print(hex_encode("Fusion"))  
    print(hex_decode("467573696f6e"))  
    print(base64_encode("Fusion"))  
    print(base64_decode("RnVzaW9u"))  
    print(format("value={0}", [42]))  
    print(serialize([1, 2, 3]))  
    print(deserialize("[1,2,3]"))  
    print(random_int(7, 7))  
    log_info("ready")
```

```
// expected output:      1970-01-01 00:00:00 UTC  
                        467573696f6e  
                        Fusion  
                        RnVzaW9u  
                        Fusion  
                        value=42  
                        [1, 2, 3]  
                        [1, 2, 3]  
                        7  
                        [INFO] ready
```
